

Teoría de la Información y Codificación

Práctica 2: Creación código con distribución de probabilidad asociada

José A. Montenegro Montes

5 de febrero de 2026

1. Enunciado

La práctica muestra los conceptos teóricos desarrollados en el tema 3.

A la hora de crear una codificación tenemos que tener en cuenta las propiedades que vimos en el tema anterior. La fuente emisora de los símbolos tiene asociada una distribución de probabilidad que la caracteriza. Dicha distribución de probabilidad debe ser tomada en cuenta a la hora de crear los códigos que realizaremos en esta práctica.

En la práctica además calcularemos un indicador, la entropía, que nos permitirá determinar el límite inferior y superior de la longitud del código que estamos buscando.

Para el correcto funcionamiento de la práctica, el alumno deberá implementar los siguientes métodos:

- *entropía*. Calcula el valor de la entropía de una distribución dada (página 23 del tema 3).
- *longitudMedia*. Calcula la longitud media de un código (página 23 del tema 3).
- *reglaShannonFano*. Obtiene los parámetros n_i para establecer un código posteriormente con el árbol binario (página 39 del tema 3).

2. Conclusiones

Con esta práctica observamos como crear códigos adaptados a una fuente generadora. La propiedad que caracteriza a la fuente es la distribución de probabilidad asociada a la emisión de los símbolos. Por tanto, es necesario tener en consideración la distribución de probabilidad a la hora de crear la codificación.

El cálculo de la entropía nos ofrece información sobre el límite inferior en la codificación que podemos obtener. Mediante la regla de Shannon-Fano podemos establecer una codificación “acceptable” pero no óptima. Para obtener la codificación óptima es necesario aplicar las dos reglas de Huffman.

La salida del código, una vez implementados los métodos correctamente, será la siguiente:

```
1
2
3 Ejercicio 5. Probabilidades: [0.4, 0.2, 0.2, 0.1, 0.1]
4
5 Shannon Fano Codigos
6 -----
7
8 Code obtained with 0 1 2 2 0 parameters:
9 00 010 011 1000 1001
10
11 Huffman Codigos
12 -----
13
14 Optimal Code obtained with [1, 1, 1, 2, 0, 0, 0, 0, 0] parameters:
15 [0, 10, 110, 1110, 1111]
16
17 ***** Calculo Valores *****
18
19 Entropia: 2.1219280948873624
20 Longitud Media: 2.8
21 Longitud Optima: 2.2
22 Verificar  $h < l_o < l_m < h+1$ : true
23
```

3. Código

Clase Fuente

```
/**
 *
 */
package Practica2;

import java.util.Arrays;

import Practica1.BinaryTreeCod;
import Practica1.TreeException;
```

```

/**
 * Practica 2. TIC
 *
 * @author Jose A Montenegro
 * @version 1.0 8/10/2013
 */
public class Fuente {

    private static final int AddLevel = 3; //ToDo: Obtener valor teorico.
    double []probabilidades;
    int lengProbabilidades=0;
    BinaryTreeCod code;
    //Arbol para calcular codigo con los parametros de shannon-fano

    //Valores para shannon-Fanno
    boolean codeSetting=false;
    boolean parametersValid=false;
    int[] parametros;
    int parametroslong;

    HuffmanCode huffmanCodification=null;

    /*
     * Constructor de la clase.
     * Inicialmente ordeno las probabilidades para estar
     * en orden decreciente.
     * Ojo puede ser un problema al codificar. Para este
     * ejercicio no supone ningun problema
     */
     * @param distribution Array de probabilidades
     */
    public Fuente( double [] distribution) {

        //Ordenan de forma decreciente las probabilidades.
        // ToDo: Sustituir por un algoritmo unico.

        probabilidades = reverseOrder(distribution);

        lengProbabilidades = probabilidades.length;

        parametroslong= (int)logBase2(lengProbabilidades)+AddLevel;
        parametros = new int[parametroslong];
    }
}

```

```

}

/**
 * Calcula la entropia
 * @return      Entropia de la distribucion de probabilidad.
 */
public double entropia() {
    double h = 0;

    // Tu codigo aqui

    return h;
}

/**
 * Calcula la longitud media de un codigo.
 *
 * Nota: El parametro n1 esta en la posicion cero del array.
 * @param  Codigo
 * @return      Longitud media.
 */
public double longitudMedia() {
    double l = 0.0;
    int j=0;
    int temp=parametros[0];

    if (parametersValid){

        // Tu codigo aqui

    }

    return l;
}

/**
 * Devuelve los parametros ni para un codigo segun la regla de Shannon-Fano.
 *
 * @return      Parametros de un codigo.
 */
public int [] reglaDeShannonFano() {

    int y;

```

```

// Tu codigo aqui

        parametersValid=true;

        return parametros;
    }

    /**
     * Calcula los codigos mediante arbol binario de practica 1 y
     * los parametros de Shannon-Fano
     *
     * @return
     * @throws TreeException
     */
    public boolean setCodeShannonFano() throws TreeException{

        int [] parameters=reglaDeShannonFano();
        int leng=parameters.length;

        code = new BinaryTreeCod(leng);
        codeSetting=code.buildCodeParameters(parameters);

        return codeSetting;
    }

    /**
     * Calcula el codigo optimo mediante la clase Huffman
     *
     * @return
     */
    public void generarCodigoOptimo(){
        huffmanCodification = new HuffmanCode(probabilidades);
    }

    /**
     * Obtengo la longitud optima del codigo huffman.
     * Si no esta calculado codigo optimo devuelve cero.
     *
     * @return longitudOptima
     */
    public double longitudOptima(){

        if(huffmanCodification==null) return 0.0;
    }

```

```

        return huffmanCodification.longitudOptima();
    }

    /**
     * Imprime el codigo optimo calculado
     */

    public void printCodeOptima(){

        if(huffmanCodification!=null) {
            int huffmanParameters[];
            huffmanParameters = huffmanCodification.getParametrosOptima();
            System.out.println("\nOptimal Code obtained with "+
                Arrays.toString(huffmanParameters)+" parameters:");
            System.out.println(huffmanCodification.getCodeOptima().toString());
        }
    }

    /**
     * Verifica desigualdades de la pagina 54
     *
     * @param entropia
     * @param lm
     * @param lo
     * @return verdadero si cumple las condiciones
     */

    public boolean Verificar (double entropia,double lm, double lo){
        if (entropia<lo & lo< lm & lm <(entropia+1)) return true;
        else return false;
    }

    /**
     * Calculo logartimo base 2 mediante cambio de base
     * @param num
     * @return
     */
    private double logBase2(double num) {
        return logCambioBase(num,2);
    }

    private double logCambioBase(double num, int b) {
        return Math.log(num)/Math.log(b);
    }

```

```

/*****
 * Ordena de forma descendente las probabilidades
 * ToDo: Mejorar algoritmo
 * @param distribution
 * @return
 */
private double [] reverseOrder(double distribution []){

    Arrays.sort(distribution);

    double temp;

    for(int i = 0; i < distribution.length / 2; i++) {
        temp = distribution[i];
        distribution[i] = distribution[distribution.length - 1 - i];
        distribution[distribution.length - 1 - i] = temp;
    }
    return distribution;
}

public static void main(String[] args) {

    double [] probabilidadesEjemplo5={0.4,0.2,0.2,0.1,0.1};

    Fuente fuente = new Fuente (probabilidadesEjemplo5);
    int [] parameters=fuente.reglaDeShannonFano();

    // Creo codigo con los parametros que calculamos con ShannonFano mediante a

    BinaryTreeCod code = new BinaryTreeCod(parameters.length);
    boolean result=false;
    try {
        result = code.buildCodeParameters(parameters);
    } catch (TreeException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }

    //Calculo codigo optimo con huffman

    fuente.generarCodigoOptimo();
}

```

```

        double entropia, lm, lo;
        entropia      = fuente.entropia();
        lm            = fuente.longitudMedia();
        lo            = fuente.longitudOptima();

System.out.println("Ejercicio 5. Probabilidades: "+Arrays.toString
    (probabilidadesEjemplo5));

System.out.println("\nShannon Fano Codigos\n-----");
    if (result) code.printCodes(parameters);

System.out.println("\n\n Huffman Codigos\n-----");
    fuente.printCodeOptima();

        System.out.println("\n***** Calculo Valores *****\n");
        System.out.println("Entropia: "+entropia);
        System.out.println("Longitud Media: "+lm);
        System.out.println("Longitud Optima: "+lo );

System.out.println("Verificar h<lo<lm<h+1: "+
    fuente.Verificar(entropia, lm, lo));

    }

}

```

Clase HuffmanCode Java.

```

package Practica2;

import java.util.*;

/**
 * Practica 2. TIC
 *
 * @author Jose A Montenegro
 * @version 1.0 8/10/2013
 *
 * Algoritmo basado en la version http://rosettacode.org/wiki/Huffman\_coding#Java

```



```

*
*/

public class HuffmanCode {
    double[] probabilidades;
    HuffmanTree tree;
    List <Values> CodeList;

    public HuffmanCode (double[] probabilidadesP){
        probabilidades=probabilidadesP;

        tree= regla1(probabilidades);
        CodeList=new ArrayList<Values>();
        CodeList=regla2(tree, new StringBuffer(),CodeList);

    }

    /**
     *
     * @param probabilidades
     * @return
     */
    public HuffmanTree regla1(double[] probabilidades) {

        PriorityQueue<HuffmanTree> trees = new PriorityQueue<HuffmanTree>();

        for (int i = 0; i < probabilidades.length; i++)
            trees.add(new HuffmanLeaf(probabilidades[i]));

        assert trees.size() > 0;

        while (trees.size() > 1) {

            //Optiene los dos elementos menores
            //Estan ordenados
            HuffmanTree a = trees.poll();
            HuffmanTree b = trees.poll();
            //Crea un nodo nuevo con los dos anteriores
            trees.offer(new HuffmanNode(a, b));

        }

        return trees.poll();

    }

    /**
     *
     * @param tree

```

```

    * @param prefix
    * @param code
    * @return
    */

    public List <Values> regla2
        (HuffmanTree tree, StringBuffer prefix, List <Values> code) {

        assert tree != null;
        Values value;

        if (tree instanceof HuffmanLeaf) {
            HuffmanLeaf leaf = (HuffmanLeaf)tree;
            value=new Values(prefix.toString(),leaf.frequency);
            code.add(value);

        } else if (tree instanceof HuffmanNode) {
            HuffmanNode node = (HuffmanNode)tree;

            prefix.append('0');
            code= regla2(node.left, prefix,code);
            prefix.deleteCharAt(prefix.length()-1);

            prefix.append('1');
            code= regla2(node.right, prefix,code);
            prefix.deleteCharAt(prefix.length()-1);

        }
        return code;
    }

    /**
     * Obtiene los codigos
     * @return
     */
    public List <Values> getCode(){
        return CodeList;
    }

    /**
     * Calcula la longitud optima
     * @return
     */
    public double longitudOptima() {
        double l=0.0;

        Iterator<Values> itr = CodeList.iterator();

```

```

        while(itr.hasNext()) {
            Values element = (Values) itr.next();
            l+=element.frecuencia*element.size;
        }
        return l;
    }

    /**
     * Obtiene los parametros optimos
     * @return
     */
    public int [] getParametrosOptima() {
        int parameters [] =new int [10];
        Iterator<Values> itr = CodeList.iterator();

        while(itr.hasNext()) {
            Values element = (Values) itr.next();
            parameters[element.size-1]++;
        }
        return parameters;
    }

    /**
     * Obtiene los codigos de huffman
     * @return
     */
    public List<String> getCodeOptima() {
        List<String> code = new ArrayList<String>();
        Iterator<Values> itr = CodeList.iterator();

        while(itr.hasNext()) {
            Values element = (Values) itr.next();
            code.add(element.code);
        }
        return code;
    }

    /**
     * Clases Auxiliares para construir el arbol Huffman
     */

    class HuffmanTree implements Comparable<HuffmanTree> {
        public double frequency; // the frequency of this tree
        public HuffmanTree(double freq) { frequency = freq; }

        // compares on the frequency
        public int compareTo(HuffmanTree tree) {

```

```

        return (int) (frequency*100 - tree.frequency*100);
    }

    public String toString(){
        return frequency+" ";
    }
}

class HuffmanLeaf extends HuffmanTree {
    public HuffmanLeaf(double freq) {
        super(freq);
    }
}

class HuffmanNode extends HuffmanTree {
    public final HuffmanTree left, right; // subtrees

    public HuffmanNode(HuffmanTree l, HuffmanTree r) {
        super(l.frequency + r.frequency);
        left = l;
        right = r;
    }
}

/*****
 * Clase que devuelve los codigos con sus probabilidades
 * y longitud, utilizadas para calcular los parametros.
 *
 * @author joseamontenegromontes
 */
class Values {
    double frecuencia;
    String code;
    int size;

    public Values(String codeA, double frecuenciaA) {
        frecuencia=frecuenciaA;
        code=codeA;
        size=codeA.length();
    }

    public String toString (){
        return frecuencia + " "+code;
    }
}

```

}

}
